

13

We Have Contact! – Networking Basics

In today's world where everyone and everything is interconnected, playing games with your friends is not a new thing anymore. It has become a standard amongst many groups. Expressions such as "frag" or "camping" became buzzwords amongst gamers. Whether it's a 2-4 player LAN party or a massive multiplayer online game, networking obviously plays a huge role in the gaming circles. Introducing the element of other human players amplifies the content added, on top of making the game's universe seem much more alive and flourishing. In many instances, this sort of phenomenon actually brings people together and provides a very enjoyable experience, as long as it's not lagging. It's about time we harness the essence of multiplayer, and perhaps even propagate the six degrees of separation.

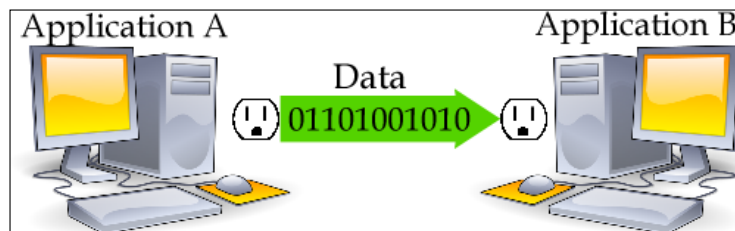
In this chapter, we're going to be covering:

- Fundamentals of networking applications
- Utilizing threads and ensuring data safety
- Implementing our own basic communication protocol
- Building a simple chat client and server

Let's break the isolation of your system and open it up to the outside world!

Basics of networking

First things first, let's cover a term that is pretty much synonymous with networking at this point: sockets. What is a socket? In its simplest terms, a socket is just an interface that is used for network communications. When two applications are communicating, at least two sockets are involved and data is exchanged between them. When data is sent from application A to application B, it first leaves from the socket of application A, travels throughout the internet, and hopefully reaches the socket of application B:



Each socket has to be bound to something referred to as a port, which can be imagined as a gateway to a system. Each gateway is used for different purposes and can only be used by one socket at a time. In the simplest terms, a port is just a 16-bit numerical value, which means a port number can be as high as 65535. While a service is using a specific port, another socket cannot bind to it until it's freed. The most commonly used ports are in a range of 20-1024. For example, port 80 is always used for HTTP traffic, which is what most website hosting servers operate on.

SFML networking

In order to access network constructs in SFML, we must first include the network header:

```
#include <SFML/Network.hpp>
```

Building a project that has networking capabilities also requires more library files in order to link properly, specifically `sfml-network.lib`, `ws2_32.lib`, and `winmm.lib`. Having these libraries included will ensure that the project compiles properly.

There are several types of sockets available out there, each with specific features, advantages, and disadvantages. SFML provides us with two basic types: TCP and UDP. **TCP** stands for **Transmission Control Protocol**, while **UDP** stands for **User Datagram Protocol**. Both of these protocols are capable of sending and receiving data, but they are fundamentally different from each other under the hood. It's fair to mention that while two sockets of the same type cannot bind to the same port, it can still be bound to by two different protocols.

TCP sockets

TCP is a connection-based protocol, which means that before data can be exchanged, a connection has to be established by having an application that attempts to initiate it (a client) connect to another application that is actively waiting for connections (a server). Let's take a look at a basic client application connection attempt:

```
sf::TcpSocket socket;
sf::Socket::Status status =
    socket.connect("192.168.1.2", 5600, sf::seconds(5.f));
if (status != sf::Socket::Done) {
    // Connection failed.
}
```

First, we create a TCP socket instance. Its `connect` method is called next, with three arguments:

- The first argument is of the type `sf::IpAddress` and is exactly what it sounds like: the IP address we are trying to connect to, which has to be open and have a server accepting connections.
- The second argument is the port number.
- Lastly, we have the third argument, which is completely optional. It's the timeout value after which the socket should give up and throw an error. If this argument isn't provided, the default operating system time-out value is used.

The return value of the `connect` method is captured and stored in a `sf::Socket::Status` type, which is just an enumeration table that has a few useful values, such as `Done`, `NotReady`, `Partial`, `Disconnected`, and `Error`. Every method of both socket types that has to do with sending or receiving data, connecting or disconnecting, returns a status that we can use for error checking.

In order to accept a connection on the server side when using TCP, a special class is used: `sf::TcpListener`. It has to be bound to a specific port and cannot send or receive any data:

```
sf::TcpListener listener;
if (listener.listen(5600) != sf::Socket::Done)
{
    // Unable to bind to port.
}

sf::TcpSocket incoming;
```

```
if (listener.accept(incoming) != sf::Socket::Done)
{
    // Failed accepting an incoming connection.
}
```

After the socket is set up, the listener's `accept` method is called. Along with `connect` and a few other methods we'll be covering down the line, it actually stops the application from continuing until a connection comes through. This is what's referred to as `blocking`. A good example of a blocking function from an STL library is `std::cin`. Why is this important? Well, to put it simply, networking operations are rather unpredictable. There is no way of knowing exactly how long a connection attempt may take, as the host on the other end could be unreachable. During that time, your application will stand still and do absolutely nothing.

After a connection finally comes through, the incoming socket can be used to communicate with the client:

```
char data[100];
// ...
if (socket.send(data, 100) != sf::Socket::Done) {
    // Sending data failed.
}
```

The `send` method has two variations: a low-level one that allows the user to send a raw array of bytes and a higher level one that uses a specialized class we're going to be covering shortly. The low-level version takes in a `void` pointer and the number of bytes it should send.

[ Keep in mind that sending data could also fail for a number of reasons. Make sure to always check the returned status for errors!]

In order to receive data on the other end, a socket needs to listen:

```
char data[100];
std::size_t received;

if (socket.receive(data, 100, received) != sf::Socket::Done)
{
    // Failed receiving data.
}
```

When raw data is sent, a large-enough buffer must be provided along with the maximum size it can contain, which is the second argument of the receive method. The third argument is the number of bytes received, which is written over when data comes in. The receive method is also blocking by default. This means that it will halt the entire program until some data comes through.

Handling multiple connections

You might have noticed by now that all of the examples above focus on just one client connecting and sending data. In today's hyper-connected world that is almost never the case, so let's take a look at a way we can handle multiple TCP sockets at the same time:

```
sf::TcpSocket socket;
// ...
sf::SocketSelector selector;
selector.add(socket);
```

The `sf::SocketSelector` class provides a way for us to block on multiple sockets, instead of just one. It monitors every single socket that is added to it for incoming data, unlike the previous examples that only dealt with a single socket.



A very important thing to keep in mind is that the socket selector does not actually store the sockets added to it, but merely points to them. This means that although a socket has been added to a selector, it still must be stored in a data container of your choosing.

To handle incoming data from multiple sockets, the `wait` method of a socket selector class is used:

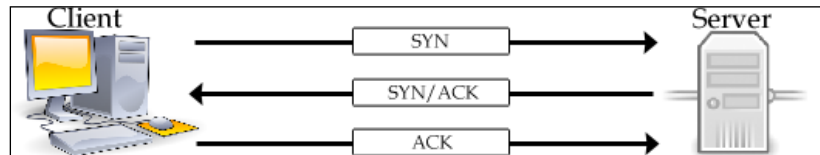
```
sf::SocketSelector selector;
std::vector<sf::TcpSocket> container;
// ...
sf::TcpSocket socket;
selector.add(socket);
container.push_back(socket);
if (selector.wait(sf::seconds(10))) {
    for (auto &itr : container) {
        if (selector.isReady(itr)) {
            // Socket received data.
            char data[100];
            std::size_t received;
            sf::Socket::Status status= itr.receive(data, 100, received);
            if (status != sf::Socket::Done) {
```

```
        // Failed receiving data...
    }
}
} else {
    // Timed out...
}
```

The argument provided in the `wait` method is, once again, optional. If one of the socket inside the selector receives something, *true* is returned and we can iterate over our data container to find the socket that received data, by using the `isReady` method.

TCP protocol specifics

One major difference between TCP and UDP is transmission reliability. The TCP protocol uses something called a three-way-handshake when establishing a connection. It looks a little something like this:



A **SYN (synchronize)** packet is first sent by the party attempting to establish a connection. The server responds with a **SYN/ACK (synchronize-acknowledgement)** packet, to which the client responds with an **ACK (acknowledgement)** packet. These three exchanges of data happen at the beginning of every connection. Afterwards, when actual data is being sent, it travels in a form of a SYN packet, to which the receiving party always replies with an ACK packet. If the party sending some data does not receive an ACK response, the same data is sent again after a specific time interval. All of this data being sent back and forth is also tagged with a sequence number, which enables the TCP protocol to also ensure that data arrives in order. This provides reliability, but at a price. Because of additional checks, waiting if some data got lost somewhere and additional data overhead, TCP ends up being slower and bulkier. If a packet gets lost somewhere, the receiving party has to wait until the same data is retransmitted in order to continue. For most applications and even certain types of games this difference in speed is negligible. However, some really fast-paced games that require highest possible efficiency and don't care about packet loss end up using UDP.

User datagram protocol

Both TCP and UDP sockets in SFML actually inherit from the same base class, which means a lot of the functionality that we saw with TCP carries over. One major difference, however, is that UDP is connectionless. This means that there is no such thing as a three-way-handshake for UDP and for that matter, no acknowledgement packets either. UDP is centered on data being sent. There are no order checks, no sequence numbers, no bloated packets, and no guarantee that what was sent out is going to reach its destination. This total elimination of error checking reduces the packet overhead from 20 bytes when using TCP all the way down to 8 bytes.

With all of that said, there are some limitations that UDP has, such as the maximum size of the data being sent out. Data in UDP is being sent out in datagrams instead of streams, which is how TCP handles it. The maximum imposed datagram size, which is a little less than 65536 bytes, cannot be exceeded.

Because UDP is connectionless, there is no equivalent of `sf::TcpListener` that can be used to accept incoming traffic. The socket must be bound to a specific port before it can be used though:

```
sf::UdpSocket socket;

// Bind to port 5600
if (socket.bind(5600) != sf::Socket::Done)
{
    // Binding failed.
}
```

Binding to a random port is also possible, thanks to `sf::Socket::AnyPort`, which can be passed into the `bind` method instead of a numerical constant. It can be retrieved later like this:

```
sf::UdpSocket socket;
// ...
unsigned int port = socket.getLocalPort();
```

Sending and receiving carries the same gist, except for having to provide additional arguments for an IP address and port that the data is being sent to or received from, on the count of UDP being connectionless:

```
sf::UdpSocket socket;
// ...
char data[100];
if (socket.send(data, 100, "192.168.1.2", 5600)
    != sf::Socket::Done)
{
    // ...
}
```

```
        // Sending failed.
    }
    // ...
    sf::IpAddress ipAddr;
    unsigned short port;
    std::size_t received;
    if (socket.receive(data, 100, received, ipAddr, port)
        != sf::Socket::Done)
    {
        // Receiving failed.
    }
```

Lastly, UDP sockets do work with a `sf::SocketSelector` class, but given the nature of UDP, there are fewer instances where that actually come in handy, because all data can simply be sent and received by using one or two sockets at most.

Alternative to sending raw data

Simply sending raw bytes across a network can get quite tricky, not to mention problematic. The first and perhaps the biggest issue is the **endianness** of a machine. Some processors interpret data in a different order than others. In a big-endian family, the most significant byte is stored first, while a little-endian family machine would do the opposite. Raw data being sent from a big-endian machine to a little-endian machine would be interpreted differently and result in funky results.

On top of data being stored differently amongst all types of machines, the sizes of basic variables in C++ may vary between different machines and compilers. If that is not enough, the TCP protocol introduces additional headaches due to it not preserving message boundaries. Chunks of data being sent out can be split and combined, which can cause problems if they are not properly re-constructed by the receivers.

While all of this sounds fairly horrific, there are solutions to all of these problems. Data type size variations can be addressed by using SFML's fixed-size types, such as `sf::Int8`, `sf::UInt16`, and so on. They are simple type definitions, mapped to data types that are sure to have the expected size depending on the platform. Exchanging these types over the network instead re-assures data safety.

SFML packets

The endianness and message boundary problems require slightly more effort to resolve. Enter `sf::Packet`! It is a specialized, lightweight class that can be used to pack/extract data. SFML packets use the exact same interface as standard streams by using the `<<` and `>>` operators for data insertion and extraction, as seen here:

```
sf::Int16 n = 16;
float f = 32.f;
std::string str = "Aloha";

sf::Packet packet;
packet << n << f << str;
// ...
packet >> n >> f >> str;
```

While packing data is always guaranteed to work, extracting it can in fact fail. If it does, the packet error flag is set. Checking whether the flag is set or not is similar to testing a Boolean value, which is again similar to standard streams:

```
if(!(packet >> n)){
    // Failed extraction.
}
```

Both TCP and UDP packets do provide overloaded send and receive methods that work with instances of `sf::Packet`:

```
sf::Packet packet;
// TCP
tcpSocket.send(packet);
tcpSocket.receive(packet);
// UDP
udpSocket.send(packet, "192.168.1.2", 5600);
sf::IpAddress senderIP;
unsigned short senderPort;
udpSocket.receive(packet, senderIP, senderPort);
```

Custom data types can also be fed into or extracted from the `sf::Packet` structure, if overloads of `<<` and `>>` operators are provided, as follows:

```
struct SomeStructure{
    sf::Int32 m_int;
    std::string m_str;
};

sf::Packet& operator <<(sf::Packet& l_packet,
    const SomeStructure& l_struct)
```

```
{
    return l_packet << l_struct.m_int << l_struct.m_str;
}

sf::Packet& operator >>(sf::Packet& l_packet,
    SomeStructure& l_struct)
{
    return l_packet >> l_struct.m_int >> l_struct.m_str;
}
```

This enables easy insertion and extraction of a custom data type:

```
SomeStructure s;
sf::Packet packet;

packet << s;
packet >> s;
```

Using SFML packets with TCP sockets poses a small restriction. Due to the message boundaries having to be preserved, some extra data is sent in the packet. This means that data sent in a form of a SFML packet has to be received by using a SFML packet. UDP does not pose this restriction, as the protocol itself preserves message boundaries.

Non-blocking sockets

Both TCP and UDP sockets, as well as the TCP listener, are blocking by default. Their blocking mode can be changed to return immediately:

```
sf::TcpSocket tcp;
tcp.setBlocking(false);

sf::TcpListener tcpListener;
tcpListener.setBlocking(false);

sf::UdpSocket udp;
udp.setBlocking(false);
```

Receiving on a non-blocking socket that has no data incoming would return `sf::Socket::NotReady`, as well as trying to accept a TCP connection if there are none pending. Putting your sockets in a non-blocking mode is the easiest way to not halt your program's execution by instead checking the availability of data or connections each time it is updated.

Non-blocking TCP sockets are not guaranteed to send all of the data you pass to it, even when using instances of `sf::Packet`. If a `sf::Socket::Partial` status is returned, the data must be sent again at the exact byte offset of where the last call to `send` stopped. If raw data is sent, make sure to use this `send` overload:

```
send(const void* data, std::size_t size, std::size_t& sent)
```

It overwrites the third provided argument with the exact number of bytes sent out.

Sending `sf::Packet` instances does not require you to keep track of the byte offset, as it is stored in the packet itself. This means that you cannot destroy the packet instance until it has been successfully sent. Creating a new packet and filling it with the exact same data will not work, as the data offset that was stored internally inside the packet is lost.

Letting the traffic flow

There are a lot more subtleties to communicating over the internet than using the right code. As we discussed previously, the port number an application uses to send or receive data can be imagined as a gateway to your system, of which there are thousands. That gateway can either be open or closed. By default, it's more likely than not that whichever port you choose to use for your program is going to be closed on your system, which doesn't matter for local connections, but anything coming from the outside world through that particular port is not going to get through. Managing your ports can be done by visiting your router's settings page. The steps required to do so are different for each router out there. Luckily, <http://portforward.com> is there to help you! By visiting it and looking up the make and model of your router on this website, you can find detailed instructions on how any port can be opened or closed.

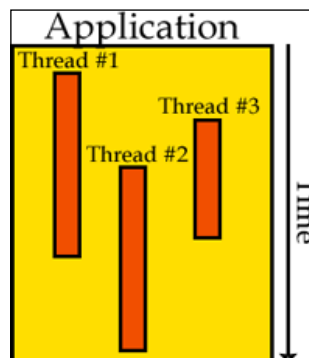


Sockets bound to `sf::Socket::AnyPort` will most likely end up binding to a port in a range of 49152 and 65535. Port forwarding works for ranges, as well as individual ports. Opening this particular range of ports will ensure that your SFML networking application works as intended when communicating over the World Wide Web.

Firewalls also tend to block this type of traffic by default. For example, the Windows firewall prompts users about allowing traffic to come through for an application that's being launched for the first time. Depending on your application, however, that prompt may never manifest due to the Windows firewall not being the most reliable piece of software ever written. If all of your key ports are open and a particular program still doesn't seem to be sending or receiving anything, make sure to add your client or server program to the "allowed list" of the Windows firewall, by going to **Control Panel**, clicking on **Windows Firewall**, selecting **Allow a program or feature through Windows Firewall** on the left side, clicking **Change settings**, and finally hitting the **Allow another program** button. This will bring up another window that can be used to add your client/server application by browsing for it and clicking on **Add** afterwards.

Multi-threading

Having blocking functions in your code can be a real nuisance. Listening for incoming network connections or data, asking users to input something into the console, or even loading game data, like textures, maps, or sounds, can block a program from executing until it's done. Have you ever wondered how certain games have a loading bar that actually moves while the data is being loaded? How can that be done with code that is executed sequentially? The answer to that is multi-threading. Your application runs all its code sequentially from top to bottom in something referred to as the main thread. It is not a program, as it can't exist by itself. Instead, a thread only runs within your application. The beauty of this is that multiple threads can exist and run all at once, which enables parallel code execution. Consider the following diagram:



Let's say that the entire application space is the main thread, and all we do here is update and render the game. The example above is running three threads in addition to that. The first thread could be used to listen for incoming network connections. **Thread #2** is responsible for loading/unloading data when a new level is opened or closed. Lastly, thread three could be waiting for console input. Even if all three threads are blocked, the application still keeps rendering! Neat!

SFML provides us with a few fundamental types that can be used to create and control threads. Let's start by first giving a thread something to do:

```
void Test(){
    for (int i = 0; i < 10; ++i){
        std::cout << i << std::endl;
    }
}
```

This is just a basic function that we want to be executed in parallel to the main thread. How can that be done? By using the `sf::Thread`!



C++ also provides its own thread class, `std::thread`, as well its own locks and mutexes. It also provides a `std::future` class template, which is useful when accessing results of asynchronous operations.



First, it must be set up properly by providing a function or a member function pointer to its constructor:

```
sf::Thread thread1(Test);
```

The thread constructor actually provides four overloads and even takes in the return value of `std::bind` and lambda expressions, which allows us to provide any number of arguments to these functions. Once the thread is set up, it must be launched in order to execute the code:

```
thread1.launch();
```

Once the function that is executed returns, its thread is automatically stopped. The `sf::Thread` class provides a `terminate` method, but it really shouldn't be used unless you know what you're doing. It can produce unwanted behavior, including local variables not being destroyed on some operating systems. Instead, your code should be designed in a way that allows a thread to stop on its own when it's no longer needed. Terminating it by hand is not safe! You have been warned.

One last method that threads provide is the `wait` method:

```
thread1.wait();
```

The thread it is called on will halt until `thread1` is finished. This could be potentially dangerous. In the case of an infinite loop or a blocking function being called in `thread1` that never unblocks, the program will hang completely.



Never destroy an instance of `sf::Thread` before it's done! This will cause the main thread to halt, as the destructor of a thread invokes its `wait` method. Your application will be stuck.

Shared data protection

Incidentally, the reason why threads are used is also the cause for most problems a user could potentially experience. Having chunks of code that run in parallel is great, but what happens if two threads attempt to read or modify the same data? In a scenario like that, crashes and data corruption are a distinct possibility. Imagine a scenario where the main thread holds a list of entities that are to be updated and rendered. So far, so good! Next, let's introduce a new thread that is going to be running network-specific code and has access to all of our entities. If this thread decided to remove an entity for whatever reason, there's a very good chance that it might happen during either the update or render cycle of the main thread. At this point, we all know too well what happens when an iterator you're using suddenly becomes invalid. Luckily, there are ways to ensure all operations in your code are thread-safe, by synchronizing them.

SFML provides us with an interesting little class called `sf::Mutex`. It stands for mutual exclusion and operates on a very basic principle of allowing only a single thread to execute certain bits of code, while making other threads wait until it's done. Let's take a look at a basic code example to help you better understand this idea:

```
sf::Mutex mutex;

void Test() {
    mutex.lock();
    for (int i = 0; i < 10; ++i) {
        std::cout << i << std::endl;
    }
    mutex.unlock();
}
```

```
int main() {
    sf::Thread thread1(Test);
    thread1.launch();

    mutex.lock();
    // Do something data-sensitive here.
    mutex.unlock();
    return 0;
}
```

The `mutex` class provides us with two methods: `lock` and `unlock`. When a mutex is locked for the first time, the thread that locked it is given priority and is allowed to continue executing the code. If another thread calls the `lock` method of the *same* mutex while it's still locked, it's not allowed to move any further until the mutex is unlocked. Once it is, the waiting is over and the second thread is allowed to continue.

Let's analyze what happens in the code example above: `thread1` is bound to the `Test` function and launched immediately. The test function locks the mutex and because it hasn't been locked yet, the loop for printing numbers begins iterating. In the meantime, our main thread reaches the `mutex.lock();` line. A few numbers may have been printed out already by this point. Because the mutex is already locked, the main thread *halts* immediately. Once the last number of the `Test` function is printed out, the `mutex.unlock();` line is reached. This enables the main thread to lock the mutex for itself and continue. If any other thread was to invoke the `lock` method of the shared mutex, it would have to wait until the main thread is through. Finally, the mutex is unlocked and any possible thread that was waiting in the background can now resume.

There is a corner-case scenario where this could potentially be dangerous. The mutex has to be unlocked in order for the main thread to continue. What if the function that is bound to a thread suddenly throws an exception? What if it returns a value, or has a branch of `if/else` statements that return separate values? The `unlock` method could be called in each branch, but that just clutters the code, not to mention it doesn't solve the exception problem. Luckily, there is a very elegant solution to that: the `sf::Lock` class. All it does is take in a reference to a mutex in its constructor, at which point it's locked, and unlocks it in its destructor. Creating an object like that on the stack would solve all of these problems, as the mutex would just get unlocked as soon as the lock object is out of scope. Let's take a look at how it can be used:

```
sf::Mutex mutex;

void Test() {
    sf::Lock lock(mutex);
    for (int i = 0; i < 10; ++i) {
        std::cout << i << std::endl;
    }
}
```

```
        if (i == 5){ return; } // mutex.unlock() called.
    }
} // mutex.unlock() called.
```

This is a much safer piece of code. Even if there was a possibility of an exception being thrown, the shared mutex would be unlocked, allowing the rest of the program to continue.

Creating a simple communication protocol

Having covered all of the basics, we're finally ready to get designing! The first choice we need to make is which protocol suits our needs better. Losing packets in a real-time application like this is not a tragedy. It's more important that data is sent and received as quickly as possible in order to update the player and all of the entities in the game. Since TCP is a slower protocol and we would not benefit from the extra measures it takes to deliver data in order, the choice is clear. User datagram protocol is the way to go.

Let's flesh out some details of the system we're going to be building by first defining some packet types that are going to be exchanged between the server and client, as well as deciding on the type of the packet identifier. This information will be held inside the `PacketTypes.h` header:

```
using PacketID = sf::Int8;
enum class PacketType{
    Disconnect = -1, Connect, Heartbeat, Snapshot,
    Player_Update, Message, Hurt, OutOfBounds
};
void StampPacket(const PacketType& l_type, sf::Packet& l_packet);
```

Note the very last element inside the `PacketType` enumeration. It isn't an actual packet type that will be sent or received. Instead, it simply exists for convenience when checking whether a packet type is valid or not, which we'll cover soon enough. In addition to the enumeration, we're also providing a function for attaching a type to a packet. It's implemented in the `PacketTypes.cpp` file:

```
void StampPacket(const PacketType& l_type, sf::Packet& l_packet){
    l_packet << PacketID(l_type);
}
```

This function simply converts the provided type argument into a specific integer data type supplied by SFML prior to feeding it into the packet instance. Using a function to do it pays off in the long run, should we ever decide to change the communication protocol by adding additional data into the packet.

Next, let's create a header file that contains the most common bits of information shared between both the client and server. We'll simply call it `NetworkDefinitions.h`:

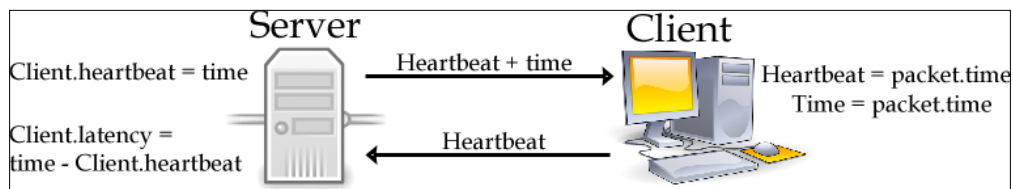
```
enum class Network{
    HighestTimestamp = 2147483647, ClientTimeout = 10000,
    ServerPort = 5600, NullID = -1
};
using ClientID = int;
using PortNumber = unsigned short;
```

These are simply all of the types and constants that both communicating sides are going to be using.

Keeping a UDP connection alive

Due to UDP sockets being connectionless, we're going to need to have a way to check if either one of the clients on the server side, or the server on the client side, has stopped responding, and therefore is deemed to have timed out. A common term for this type of mechanism is **heartbeat**. How it's implemented may differ from application to application, as well as your own sensibilities. In this case, we're going to cover a fairly basic strategy of not only maintaining a live connection, but also measuring the network delay between both sides.

For this purpose, it's always best to have the server initiate the heartbeat. It has two major benefits: less data being exchanged and a reduced risk of cheating. Let's take a look at the most conservative implementation of a server-client heartbeat:



The server is going to keep track of when the last heartbeat to a client was sent. That combined with a predefined heartbeat interval enables us to send them at a constant rate. When it is time to send one, a heartbeat packet is constructed and the local server time is attached to it. It is then sent out to the client.

The client side of this operation is much simpler. It is always waiting for a heartbeat and when one is received, the local time of the client is updated and kept around for checking timeouts later. A response to the server is then sent back, acknowledging that a heartbeat was indeed received.

When our server receives a heartbeat response from the client, the delay between these two machines is measured by subtracting the time of when the last heartbeat packet was sent out from the current time. This delay, also known as latency, is the time it takes for data to make a round trip between two hosts.

Having a heartbeat mechanism in place ensures that we are not keeping around any clients that could've potentially disconnected and thereby wasting bandwidth by sending data to unreachable hosts.

Designing the client class

With all of the things happening on the client side, be it rendering sprites or playing sounds or processing user input, it only makes sense to have all of the networking code localized inside a single class. This will allow us to communicate with the server quickly and easily. Let's begin designing that class, by first taking a look at some necessary definitions inside the `Client.h` header:

```
#define CONNECT_TIMEOUT 5000 // Milliseconds.

class Client;
using PacketHandler = std::function<
    void(const PacketID&, sf::Packet&, Client*)>;
```

The first definition is the amount of milliseconds that it takes for a client to realize that it's no longer connected to a server. This value can obviously be tweaked at any time. Following that is a definition of a function type that will be used to handle packets on the client side. We're going to be providing the client class with a pointer to a function that is responsible for handling most of the incoming information.

With that out of the way, we can begin shaping the client class:

```
class Client{
public:
    Client();
    ~Client();
    bool Connect();
    bool Disconnect();
    void Listen();
    bool Send(sf::Packet& l_packet);
    const sf::Time& GetTime() const;
```

```

const sf::Time& GetLastHeartbeat() const;
void SetTime(const sf::Time& l_time);
void SetServerInformation(const sf::IpAddress& l_ip,
    const PortNumber& l_port);

template<class T>
void Setup(void(T::*l_handler)
    (const PacketID&, sf::Packet&, Client*), T* l_instance)
{
    m_packetHandler = std::bind(l_handler, l_instance,
        std::placeholders::_1, std::placeholders::_2,
        std::placeholders::_3);
}

void Setup(void(*l_handler)(const PacketID&,
    sf::Packet&, Client*));
void UnregisterPacketHandler();
void Update(const sf::Time& l_time);
bool IsConnected() const;
void SetPlayerName(const std::string& l_name);
sf::Mutex& GetMutex();
private:
    std::string m_playerName;

    sf::UdpSocket m_socket;
    sf::IpAddress m_serverIp;
    PortNumber m_serverPort;
    PacketHandler m_packetHandler;
    bool m_connected;
    sf::Time m_serverTime;
    sf::Time m_lastHeartbeat;

    sf::Thread m_listenThread;
    sf::Mutex m_mutex;
};

```

There are a few things to note here. First, we're going to want to support both regular functions and member functions to serve as packet handlers, hence two Setup methods. The first one obviously has to be implemented in the header file due to having a template argument.

Second, this class keeps and manages its own instances of `sf::Mutex` and `sf::Thread`. This way, we can provide a common interface of thread synchronization to the outside code.

Implementing the client

With the class definition covered, it's time to actually make it do something, starting with the constructor and destructor:

```
Client::Client():m_listenThread(&Client::Listen, this){}
Client::~Client(){ m_socket.unbind(); }
```

In the client constructor's initializer list, we bind the listening thread to the `Listen` method of this class. Threads do not have a default empty constructor, which is why this is necessary. The destructor is simply used to unbind the socket we're using.

Now, let's take a stab at implementing the connection protocol:

```
bool Client::Connect(){
    if (m_connected){ return false; }
    m_socket.bind(sf::Socket::AnyPort);
    sf::Packet p;
    StampPacket(PacketType::Connect, p);
    p << m_playerName;
    if (m_socket.send(p, m_serverIp, m_serverPort) !=
        sf::Socket::Done)
    {
        m_socket.unbind();
        return false;
    }
    m_socket.setBlocking(false);
    p.clear();
    sf::IpAddress recvIP;
    PortNumber recvPORT;
    sf::Clock timer;
    timer.restart();
    while (timer.getElapsedTime().asMilliseconds() < CONNECT_TIMEOUT){
        sf::Socket::Status s = m_socket.receive(p, recvIP, recvPORT);
        if (s != sf::Socket::Done){ continue; }
        if (recvIP != m_serverIp){ continue; }
        PacketID id;
        if (!(p >> id)){ break; }
        if ((PacketType)id != PacketType::Connect){ continue; }
        m_packetHandler(id, p, this);
        m_connected = true;
        m_socket.setBlocking(true);
        m_lastHeartbeat = m_serverTime;
        m_listenThread.launch();
        return true;
    }
```

```

    }
    std::cout << "Connection attempt failed! Server info: "
        << m_serverIp << ":" << m_serverPort << std::endl;
    m_socket.unbind();
    m_socket.setBlocking(true);
    return false;
}

```

The first and most obvious step is checking whether we're already connected to the server by checking out the `m_connected` data member.

Next, the socket we're using must be bound to a port. A specific port number could be used here, but that would limit the number of connections a single computer can have at the same time. You cannot bind a socket to the same port on the same protocol twice. By using `sf::Socket::AnyPort`, we're letting SFML pick a random port that isn't being used.

In order to establish a connection, the client must first send something to the server. Since SFML already provides an excellent helper class for easy data transfer, `sf::Packet`, we're going to be taking full advantage of it.

After assigning the type `Connect` to our packet, we also write in the player name and send the packet to the server.

The rest of the code is responsible for correctly handling timeouts. First, we set our socket to be in a non-blocking mode, because we're going to be handling this in a single thread. After clearing the packet we just sent out in order to use it again, a few local variables are set up in order to capture the IP address and port number of a response. Additionally, a clock is set up in order to help us determine whether we've been waiting too long for a response.

Next, the code loops as long as the timer stays underneath the predefined time-out value, `CONNECT_TIMEOUT`. In each iteration, we invoke the receive method of our socket and capture its status. If the returned status doesn't signify success or the IP address it was received from does not match that of our server, we simply skip the current iteration. Nobody wants to receive data from an unknown source!

After verifying that the packet contains an ID and that it matches `Connect`, we pass the received information to the packet handler, set the `m_connected` flag to `true`, put the socket back into blocking mode, set the last heartbeat value to the current time, launch the listening thread, and return `true` to show success. However, if the time to successfully connect runs out, the loop is ended, an error message is printed, and the socket is unbound and set to blocking mode again.

Once the client is connected to the server, the listening thread is launched. Let's take a look at what makes it tick:

```
void Client::Listen(){
    sf::Packet packet;
    sf::IpAddress recvIP;
    PortNumber recvPORT;
    while (m_connected){
        packet.clear();
        sf::Socket::Status status =
            m_socket.receive(packet, recvIP, recvPORT);
        if (status != sf::Socket::Done){
            if (m_connected){
                std::cout << "Failed receiving a packet from "
                    << recvIP << ":" << recvPORT << ". Status: "
                    << status << std::endl;
                continue;
            } else {
                std::cout << "Socket unbound." << std::endl;
                break;
            }
        }
        if (recvIP != m_serverIp){
            // Ignore packets not sent from the server.
            continue;
        }
        PacketID p_id;
        if (!(packet >> p_id)){
            // Non-conventional packet.
            continue;
        }
        PacketType id = (PacketType)p_id;
        if (id < PacketType::Disconnect || id >= PacketType::OutOfBounds){
            // Invalid packet type.
            continue;
        }

        if (id == PacketType::Heartbeat){
            sf::Packet p;
            StampPacket(PacketType::Heartbeat, p);
            if (m_socket.send(p, m_serverIp, m_serverPort) !=
                sf::Socket::Done)
            {
                std::cout << "Failed sending a heartbeat!" << std::endl;
            }
        }
    }
}
```

```

    }
    sf::Int32 timestamp;
    packet >> timestamp;
    SetTime(sf::milliseconds(timestamp));
    m_lastHeartbeat = m_serverTime;
} else if(m_packetHandler){
    m_packetHandler((PacketID)id, packet, this); // Handle.
}
}
}

```

After setting up some local variables to hold the packet, IP, and port information, the listening thread loop is entered. It runs as long as the client is connected to the server. Each time the loop iterates, the packet instance is cleared in order to receive new data. The status of the socket receive method is stored in a local variable, `status`, and checked for success. Because the socket is in blocking mode, the listening thread will halt at the `m_socket.receive(...)` line until some data comes in.

If the returned status denotes some kind of failure, an appropriate error message is printed, presuming that the client is still connected to the server. If it's not, the socket was unbound and the loop is immediately stopped so the thread can safely terminate.

Given that some data was properly received, the originating IP address is checked. If it does not match the IP of our server, the data is discarded by skipping the current loop iteration. Similarly enough, if we're unable to extract the packet ID, or if it does not fit within our predetermined boundaries, the same outcome takes place. We don't want any malformed or unwelcome packets.

Next, we check the ID of the packet that was just received. In this particular class, we only want to worry about a single type of packets: `PACKET_HEARTBEAT`. These are little messages that the server sends to all clients for two reasons: time synchronization and maintaining a valid connection. Due to unforeseen circumstances, time on the server side and the client side can start going out of sync, which eventually can cause serious problems. Overwriting the client time with a timestamp coming from the server every so often eliminates this problem. In addition to that, this is how both the client and server keep track of whether the connection is still alive or not. In the case of our client, `m_lastHeartbeat` holds the latest timestamp received from the server, which can be checked for timeouts later.

If the packet ID is something else, it simply gets passed into the packet handler function to be processed by a different class.

Now that we have all these ways of opening and maintaining a connection to the server, let's take a look at how it can be terminated:

```
bool Client::Disconnect() {
    if (!m_connected) { return false; }
    sf::Packet p;
    StampPacket(PacketType::Disconnect, p);
    sf::Socket::Status s =
        m_socket.send(p, m_serverIp, m_serverPort);
    m_connected = false;
    m_socket.unbind(); // Unbind to close the listening thread.
    if (s != sf::Socket::Done) { return false; }
    return true;
}
```

First, the state of the client is checked. We don't need to disconnect if there is no connection to begin with. A packet instance is then constructed with a type of `Disconnect` and sent to the server. After the `m_connected` flag is set to `false`, we unbind our socket and return `true` or `false`, based on whether sending the packet was successful or not.



When a socket is in blocking mode, its receive method waits until some data arrives before continuing. Having something like that happen in a separate thread would leave it running, and therefore stop our program from quitting. One way to prevent that is by unbinding a socket that is being used. It makes the receive method return an error, which we have handled in the `Listen` method of our client class.

Sending data to the server is quite simple, as this next method shows:

```
bool Client::Send(sf::Packet& l_packet) {
    if (!m_connected) { return false; }
    if (m_socket.send(l_packet, m_serverIp, m_serverPort) !=
        sf::Socket::Done)
    {
        return false;
    }
    return true;
}
```

We take in a reference to an existing packet that needs to be sent out. The method immediately returns `false` if we're not connected to the server or if the `send` method of our socket returns anything else than `sf::Socket::Done`.

We also need a way to provide a custom packet handler function that is to be used by this class. The member function version of this method was already implemented in the header file, and all that's left is handling a function pointer version:

```
void Client::Setup(void(*l_handler)
    (const PacketID&, sf::Packet&, Client*))
{
    m_packetHandler = std::bind(l_handler,
        std::placeholders::_1, std::placeholders::_2,
        std::placeholders::_3);
}
```

For balance, every positive needs a negative. Let's provide a way to remove any ties to functions that may no longer exist once the code starts wrapping up:

```
void Client::UnregisterPacketHandler() {
    m_packetHandler = nullptr;
}
```

Last but not least, the update method:

```
void Client::Update(const sf::Time& l_time) {
    if (!m_connected) { return; }
    m_serverTime += l_time;
    if (m_serverTime.asMilliseconds() < 0) {
        m_serverTime -= sf::milliseconds(
            sf::Int32(Network::HighestTimestamp));
        m_lastHeartbeat = m_serverTime;
        return;
    }
    if (m_serverTime.asMilliseconds() -
        m_lastHeartbeat.asMilliseconds() >=
        sf::Int32(Network::ClientTimeout))
    {
        // Timeout.
        std::cout << "Server connection timed out!" << std::endl;
        Disconnect();
    }
}
```

The main purpose of this method is keeping track of passing time by adding the time between updates to the server time.

Now, you might notice something strange in the next few lines. Why are we checking for the server time being below zero? Well, the amount of milliseconds that have passed since the beginning is represented by a signed 32-bit integer. Its maximum positive value is 2,147,483,647, after which it goes right to the negatives, -2,147,483,648 to be precise. Granted, this doesn't happen very often. In fact, the server would have to be running continuously for almost 25 whole days for the timestamp to reach the values we're talking about. Still, a corner-case scenario is not worthy of ignoring simply because it's unlikely to happen. Subtracting the highest possible value of a 32-bit integer from the server timestamp "wraps" the time around back into the positive realm and allows it to continue like nothing happened.

The update method is also where we check for our connection timing out. If the difference between current time and the last heartbeat received from the server is greater or equal to the timeout value in milliseconds, the `Disconnect` method is invoked.

The server class

Now, it's time to take a look at the way things are handled on the other side of the wire. Let's begin by defining some constants:

```
#define HEARTBEAT_INTERVAL 1000 // Milliseconds.
#define HEARTBEAT_RETRIES 5
```

Our server application is going to send out heartbeats every second and retry five times before it times out a client. Speaking of clients, there's additional information that we need to keep track of, which calls for a good data structure that holds it all:

```
struct ClientInfo{
    sf::IpAddress m_clientIP;
    PortNumber m_clientPORT;
    sf::Time m_lastHeartbeat;
    sf::Time m_heartbeatSent;
    bool m_heartbeatWaiting;
    unsigned short m_heartbeatRetry;
    unsigned int m_latency;

    ClientInfo(const sf::IpAddress& l_ip, const PortNumber& l_port,
        const sf::Time& l_heartbeat): m_clientIP(l_ip),
        m_clientPORT(l_port), m_lastHeartbeat(l_heartbeat),
        m_heartbeatWaiting(false), m_heartbeatRetry(0), m_latency(0)
    {}
}
```

```

ClientInfo& operator=(const ClientInfo& l_rhs){
    m_clientIP      = l_rhs.m_clientIP;
    m_clientPORT    = l_rhs.m_clientPORT;
    m_lastHeartbeat  = l_rhs.m_lastHeartbeat;
    m_heartbeatSent  = l_rhs.m_heartbeatSent;
    m_heartbeatWaiting = l_rhs.m_heartbeatWaiting;
    m_heartbeatRetry  = l_rhs.m_heartbeatRetry;
    m_latency        = l_rhs.m_latency;
    return *this;
}
};

```

Besides simply keeping track of the IP and port of a client, we also need to know when the last heartbeat was sent to them, whether the server is waiting for a heartbeat response, the number of heartbeat retries made, and the current latency the client has.



Keeping track of latency provides tons of potential benefits, ranging from assessing quality of service and accurate matchmaking to maximizing the accuracy of a network simulation.

Next, the data types we're going to be using throughout the `Server` class deserve a look:

```

using Clients = std::unordered_map<ClientID, ClientInfo>;
class Server;
using PacketHandler = std::function<void(sf::IpAddress&,
    const PortNumber&, const PacketID&, sf::Packet&, Server*)>;
using TimeoutHandler = std::function<void(const ClientID&)>;

```

As you can see, this class also uses a custom function that is going to be handling incoming packets. In addition to that, we're going to need to be able to process client timeouts outside of this class as well, which can also be done through means of using a function pointer.

We have everything we need, so let's write the `Server` class header:

```

class Server{
public:
    template <class T>
    Server(void(T::*l_handler)(sf::IpAddress&, const PortNumber&,
        const PacketID&, sf::Packet&, Server*),
        T* l_instance): m_listenThread(&Server::Listen, this)
    {

```

```
        m_packetHandler = std::bind(l_handler, l_instance,
            std::placeholders::_1, std::placeholders::_2,
            std::placeholders::_3, std::placeholders::_4,
            std::placeholders::_5);
    }

    Server(void(*l_handler)(sf::IpAddress&, const PortNumber&,
        const PacketID&, sf::Packet&, Server*));
    ~Server();

    template<class T>
    void BindTimeoutHandler(void(T::*l_handler)
        (const ClientID&, T* l_instance)
    {
        m_timeoutHandler = std::bind(l_handler, l_instance,
            std::placeholders::_1);
    }
    void BindTimeoutHandler(void(*l_handler)(const ClientID&));

    bool Send(const ClientID& l_id, sf::Packet& l_packet);
    bool Send(sf::IpAddress& l_ip, const PortNumber& l_port,
        sf::Packet& l_packet);
    void Broadcast(sf::Packet& l_packet,
        const ClientID& l_ignore = ClientID(Network::NullID));

    void Listen();
    void Update(const sf::Time& l_time);

    ClientID AddClient(const sf::IpAddress& l_ip,
        const PortNumber& l_port);
    ClientID GetClientID(const sf::IpAddress& l_ip,
        const PortNumber& l_port);
    bool HasClient(const ClientID& l_id);
    bool HasClient(const sf::IpAddress& l_ip,
        const PortNumber& l_port);
    bool GetClientInfo(const ClientID& l_id, ClientInfo& l_info);
    bool RemoveClient(const ClientID& l_id);
    bool RemoveClient(const sf::IpAddress& l_ip,
        const PortNumber& l_port);

    void DisconnectAll();
    bool Start();
    bool Stop();
```

```

    bool IsRunning();

    unsigned int GetClientCount();
    std::string GetClientList();

    sf::Mutex& GetMutex();
private:
    void Setup();
    ClientID m_lastID;

    sf::UdpSocket m_incoming;
    sf::UdpSocket m_outgoing;

    PacketHandler m_packetHandler;
    TimeoutHandler m_timeoutHandler;

    Clients m_clients;
    sf::Time m_serverTime;

    bool m_running;

    sf::Thread m_listenThread;
    sf::Mutex m_mutex;

    size_t m_totalSent;
    size_t m_totalReceived;
};

```

Just like the `Client` class, we want to support both member functions and regular functions being bound as packet and timeout handlers. Additionally, we also need an instance of a mutex and two sockets: one for listening and one for sending data. Having two sockets on the server side provides separation between different operations, which sometimes may result in runtime errors and data corruption. For an added bonus, we're also keeping track of all data sent and received.

Implementing server

Let's start by taking a look at the second constructor and the destructor of this class:

```

Server::Server(void(*l_handler)(sf::IpAddress&, const PortNumber&,
    const PacketID&, sf::Packet&, Server*))
    : m_listenThread(&Server::Listen, this)
{
    // Bind a packet handler function.

```

```
m_packetHandler = std::bind(l_handler,
    std::placeholders::_1, std::placeholders::_2,
    std::placeholders::_3, std::placeholders::_4,
    std::placeholders::_5);
}

Server::~Server() { Stop(); }
```

Nothing too interesting is happening here. The constructor simply binds the provided packet handler function, while the destructor just invokes the `Stop` method, which we are going to cover shortly. Speaking of binding, we also need a function to handle client timeouts:

```
void Server::BindTimeoutHandler(void(*l_handler)
    (const ClientID&))
{
    m_timeoutHandler = std::bind(l_handler, std::placeholders::_1);
}
```

Simply having the client get disconnected is not always enough, depending on your application. One of the main ways we're going to be taking advantage of this feature is de-spawning entities in the game world.

Now, let's take a peek at how we start our server:

```
bool Server::Start() {
    if (m_running) { return false; }
    if (m_incoming.bind(SERVER_PORT) != sf::Socket::Done) {
        return false;
    }
    m_outgoing.bind(sf::Socket::AnyPort);
    Setup();
    std::cout << "Incoming port: " <<
        m_incoming.getLocalPort() << ". Outgoing port: "
        << m_outgoing.getLocalPort() << std::endl;
    m_listenThread.launch();
    m_running = true;
    return true;
}
```

If the server is already running, or if we failed to bind the incoming socket to the predesignated port number, `false` is returned. Otherwise, the outgoing socket is bound to a random port, a `Setup` method is invoked to set up some data members, the listening thread is launched, and the `m_running` flag is set to `true`. You can stop the server like this:

```
bool Server::Stop(){
    if (!m_running){ return false; }
    DisconnectAll();
    m_running = false;
    m_incoming.unbind(); // Stops the listening thread.
    return true;
}
```

If the server is actually running, a `DisconnectAll` method is invoked to drop all the clients. The running flag is then set to `false` and the incoming socket is unbound, which in turn stops the listening thread from running, as the socket is in blocking mode.

Here's a little helper method that initializes some data members to their default states:

```
void Server::Setup(){
    m_lastID = 0;
    m_running = false;
    m_totalSent = 0;
    m_totalReceived = 0;
}
```

This is invoked every time a server is started, as demonstrated previously.

Sending data to clients is fairly straightforward, as you can see here:

```
bool Server::Send(const ClientID& l_id, sf::Packet& l_packet){
    sf::Lock lock(m_mutex);
    auto itr = m_clients.find(l_id);
    if (itr == m_clients.end()){ return false; }
    if (m_outgoing.send(l_packet, itr->second.m_clientIP,
        itr->second.m_clientPORT) != sf::Socket::Done)
    {
        std::cout << "Error sending a packet..." << std::endl;
        return false;
    }
    m_totalSent += l_packet.getDataSize();
    return true;
}
```

With a little bit of STL find magic, we retrieve the client's information from the container it's stored in and send the packet out.



Note the mutex lock in the very first line. This is done to ensure that the client isn't removed from the container right in the middle of a send operation.

As a side bonus, it's also nice to keep track of how much data was sent and received. Here, we make use of the `getDataSize` method that `sf::Packet` provides to do just that.

For convenience, we could also write an overloaded version of the `Send` method that doesn't require a client:

```
bool Server::Send(sf::IpAddress& l_ip,
    const PortNumber& l_port, sf::Packet& l_packet)
{
    if (m_outgoing.send(l_packet, l_ip, l_port) != sf::Socket::Done)
    {
        return false;
    }
    m_totalSent += l_packet.getDataSize();
    return true;
}
```

In plenty of occasions, sending data out to only one client isn't enough. Broadcasting it to all of the clients currently connected can be useful for sending out anything from chat messages to entity states. Let's write it:

```
void Server::Broadcast(sf::Packet& l_packet,
    const ClientID& l_ignore)
{
    sf::Lock lock(m_mutex);
    for (auto &itr : m_clients)
    {
        if (itr.first != l_ignore){
            if (m_outgoing.send(l_packet, itr.second.m_clientIP,
                itr.second.m_clientPORT) != sf::Socket::Done)
            {
                std::cout << "Error broadcasting a packet to client: "
                    << itr.first << std::endl;
                continue;
            }
        }
    }
}
```

```

        m_totalSent += l_packet.getDataSize();
    }
}
}

```

Once again, it's quite basic. The client container is iterated over and each client's ID is checked for matching the `l_ignore` argument, which can be used to specify a client ID that shouldn't receive the packet being broadcasted. If data is successfully sent out, its size is added to the sent data counter.

Much like the client, our server is going to need a separate thread for processing incoming data. Let's take a look at the `Listen` method:

```

void Server::Listen(){
    sf::IpAddress ip;
    PortNumber port;
    sf::Packet packet;
    while (m_running){
        packet.clear();
        sf::Socket::Status status =
            m_incoming.receive(packet, ip, port);
        if (status != sf::Socket::Done){
            if (m_running){
                std::cout << "Error receiving a packet from: "
                    << ip << ":" << port << ". Code: " <<
                    status << std::endl;
                continue;
            } else {
                std::cout << "Socket unbound." << std::endl;
                break;
            }
        }
    }

    m_totalReceived += packet.getDataSize();

    PacketID p_id;
    if (!(packet >> p_id)){
        continue;
    } // Non-conventional packet.
    PacketType id = (PacketType)p_id;
    if (id < PacketType::Disconnect || id >= PacketType::OutOfBounds){
        continue;
    } // Invalid packet type.
}

```

```
    if (id == PacketType::Heartbeat){
        sf::Lock lock(m_mutex);
        for (auto &itr : m_clients){
            if (itr.second.m_clientIP != ip ||
                itr.second.m_clientPORT != port)
            {
                continue;
            }
            if (!itr.second.m_heartbeatWaiting){
                std::cout << "Invalid heartbeat packet received!"
                    << std::endl;
                break;
            }
            itr.second.m_ping = m_serverTime.asMilliseconds() -
                itr.second.m_heartbeatSent.asMilliseconds();
            itr.second.m_lastHeartbeat = m_serverTime;
            itr.second.m_heartbeatWaiting = false;
            itr.second.m_heartbeatRetry = 0;
            break;
        }
    } else if (m_packetHandler){
        m_packetHandler(ip, port, (PacketID)id, packet, this);
    }
}
```

As you can see, it's quite similar to the way a client listener is implemented. After some local variables are set up for capturing incoming data, we enter a while loop, during which the packet is cleared and the blocking receive method of our incoming socket is invoked, capturing the status as well. Just as before, we break from the loop if the server is no longer running and the return status from the `receive` method was anything but `sf::Socket::Done`.

After all of the packet ID checks, we get to the heartbeat part of the code. A flag is set up to indicate whether a client that sent the heartbeat was located or not.



Note that we lock our mutex here, because we're about to start iterating over the list of clients in order to find the one that sent in the heartbeat response.

If the client with matching information has been found, we also check if the server is currently waiting for a heartbeat response from them. We only want clients to be able to send in heartbeat responses in order to both accurately measure latency and prevent potential cheating attempts.

Given that this is a valid heartbeat response, latency is calculated by subtracting the time a heartbeat was sent to this particular client from the current server time. The current timestamp is also stored in `m_lastHeartbeat`, which we'll be using to determine when to dispatch the next one. Afterwards, the heartbeat waiting flag is set to `false` and the retry counter is set back to 0.

Next, let's implement adding clients:

```
ClientID Server::AddClient(const sf::IpAddress& l_ip,
    const PortNumber& l_port)
{
    sf::Lock lock(m_mutex);
    for (auto &itr : m_clients){
        if (itr.second.m_clientIP == l_ip &&
            itr.second.m_clientPORT == l_port)
        {
            return ClientID(Network::NullID);
        }
    }
    ClientID id = m_lastID;
    ClientInfo info(l_ip, l_port, m_serverTime);
    m_clients.insert(std::make_pair(id, info));
    ++m_lastID;
    return id;
}
```

Once again, since we're modifying client data, we want to lock our mutex to ensure no other piece of code running in another thread will attempt to read or modify the data as well. Following that, we do a quick search through the client container and return -1 if a specified IP and port combination already exists. Otherwise, a new client ID is assigned and the client information is inserted into the container, followed by an increment operation of `m_lastID`.

Sometimes, we may need to obtain the client ID of a client by providing their IP address and port number. Let's write a way to do just that:

```
ClientID Server::GetClientID(const sf::IpAddress& l_ip,
    const PortNumber& l_port)
{
    sf::Lock lock(m_mutex);
    for (auto &itr : m_clients){
        if (itr.second.m_clientIP == l_ip &&
            itr.second.m_clientPORT == l_port)
        {
            return itr.first;
        }
    }
}
```

```
    }  
  }  
  return ClientID(Network::NullID);  
}
```

As always, it simply iterates over each client and checks if their information matches the provided arguments. This is another instance where we need to lock the mutex in order to safely access this data.

Next up, we need some setters and getters:

```
bool Server::HasClient(const ClientID& l_id){  
    return (m_clients.find(l_id) != m_clients.end());  
}  
  
bool Server::HasClient(const sf::IpAddress& l_ip,  
    const PortNumber& l_port)  
{  
    return(GetClientID(l_ip, l_port) >= 0);  
}  
  
bool Server::IsRunning(){ return m_running; }  
sf::Mutex& Server::GetMutex(){ return m_mutex; }
```

A way of obtaining client information from a client ID is also necessary:

```
bool Server::GetClientInfo(const ClientID& l_id,  
    ClientInfo& l_info)  
{  
    sf::Lock lock(m_mutex);  
    for (auto &itr : m_clients){  
        if (itr.first == l_id){  
            l_info = itr.second;  
            return true;  
        }  
    }  
    return false;  
}
```

In this case, the provided reference to a `ClientInfo` structure is simply overwritten with the located information. It can be done in a single line of code, thanks to the overloaded assignment operator that `ClientInfo` provides. Once again, the mutex is locked because we're accessing data that can be removed or overwritten in the middle of the search otherwise.

When a client is no longer needed, it must be removed. For convenience, we provide two variations of the same method:

```
bool Server::RemoveClient(const ClientID& l_id){
    sf::Lock lock(m_mutex);
    auto itr = m_clients.find(l_id);
    if (itr == m_clients.end()){ return false; }
    sf::Packet p;
    StampPacket(PacketType::Disconnect, p);
    Send(l_id, p);
    m_clients.erase(itr);
    return true;
}
```

The first one simply locates client information by using the `find` method of a container it's stored in. If one was found, a disconnect packet is created and sent to the client before it gets erased. The second variation varies in its search method, but carries out the same basic idea:

```
bool Server::RemoveClient(const sf::IpAddress& l_ip,
    const PortNumber& l_port)
{
    sf::Lock lock(m_mutex);
    for (auto itr = m_clients.begin();
        itr != m_clients.end(); ++itr)
    {
        if (itr->second.m_clientIP == l_ip &&
            itr->second.m_clientPORT == l_port)
        {
            sf::Packet p;
            StampPacket(PacketType::Disconnect , p);
            Send(itr->first, p);
            m_clients.erase(itr);
            return true;
        }
    }
    return false;
}
```

Once again, the mutex is locked in both of them as data is being read and modified. Speaking of removing clients, how about having a method that kicks all of them out at the same time?

```
void Server::DisconnectAll(){
    if (!m_running){ return; }
    sf::Packet p;
    StampPacket(PacketType::Disconnect, p);
```

```
Broadcast(p);  
sf::Lock lock(m_mutex);  
m_clients.clear();  
}
```

It's quite a simple little bit of code. If the server is running, a disconnect packet is created, just like before, except it's broadcasted to all clients instead of just one. The mutex is then locked, right before the client container is cleared completely.

Last but definitely not least, here's the update method:

```
void Server::Update(const sf::Time& l_time){  
    m_serverTime += l_time;  
    if (m_serverTime.asMilliseconds() < 0){  
        m_serverTime -= sf::milliseconds(HIGHEST_TIMESTAMP);  
        sf::Lock lock(m_mutex);  
        for (auto &itr : m_clients)  
        {  
            itr.second.m_lastHeartbeat =  
                sf::milliseconds(std::abs(  
                    itr.second.m_lastHeartbeat.asMilliseconds() -  
                    HIGHEST_TIMESTAMP));  
        }  
    }  
}  
  
sf::Lock lock(m_mutex);  
for (auto itr = m_clients.begin(); itr != m_clients.end();){  
    sf::Int32 elapsed =  
        m_serverTime.asMilliseconds() -  
        itr->second.m_lastHeartbeat.asMilliseconds();  
    if (elapsed >= HEARTBEAT_INTERVAL){  
        if (elapsed >= CLIENT_TIMEOUT  
            || itr->second.m_heartbeatRetry > HEARTBEAT_RETRIES)  
        {  
            // Remove client.  
            std::cout << "Client " <<  
                itr->first << " has timed out." << std::endl;  
            if (m_timeoutHandler){ m_timeoutHandler(itr->first); }  
            itr = m_clients.erase(itr);  
            continue;  
        }  
        if (!itr->second.m_heartbeatWaiting || (elapsed >=  
            HEARTBEAT_INTERVAL * (itr->second.m_heartbeatRetry + 1)))  
        {  
            // Heartbeat
```

```

        if (itr->second.m_heartbeatRetry >= 3){
            std::cout << "Re-try(" << itr->second.m_heartbeatRetry
                << ") heartbeat for client "
                << itr->first << std::endl;
        }
        sf::Packet Heartbeat;
        StampPacket(PACKET_HEARTBEAT, Heartbeat);
        Heartbeat << m_serverTime.asMilliseconds();
        Send(itr->first, Heartbeat);
        if (itr->second.m_heartbeatRetry == 0){
            itr->second.m_heartbeatSent = m_serverTime;
        }
        itr->second.m_heartbeatWaiting = true;
        ++itr->second.m_heartbeatRetry;

        m_totalSent += Heartbeat.getDataSize();
    }
}
++itr;
}
}

```

Similar to the client, the server also has to worry about the timestamp running out of bounds. Unlike the client, however, we need to reset the heartbeats of every client the server has, hence the mutex lock. Speaking of which, we also need to lock the mutex right before all of the update code, as it can potentially modify any of the clients, just like any other piece of code that runs in a different thread.

After the mutex lock, we begin iterating over clients and measuring the time elapsed between now and the last heartbeat. If this time exceeds or is equal to the interval we want heartbeats to be sent out at, we first check if it either also exceeds the timeout interval or if the number of heartbeat retries has exceeded the designated value. If so, the timeout handler is invoked and the client is erased from the container.

The heartbeat code itself is quite simple. If the server isn't waiting for a reply from the client, or if it's time to retry sending another heartbeat, a packet is constructed, the server time is attached to it, and it is sent out. If this is a first try sending it, the server time also gets stored inside the `m_heartbeatSent` data member of a client entry.

A simple chat application

We have the basic framework in place to handle connections, so let's build something with it! How about a neat little console-based chat program? Let's start with the server by creating a separate project and a new file called `Server_Main.cpp`. The first thing we're going to need is a packet handler:

```
void Handler(sf::IpAddress& l_ip, const PortNumber& l_port,
            const PacketID& l_id, sf::Packet& l_packet, Server* l_server)
{
    ClientID id = l_server->GetClientID(l_ip, l_port);
    if (id >= 0){
        if ((PacketType)l_id == PacketType::Disconnect){
            l_server->RemoveClient(l_ip, l_port);
            sf::Packet p;
            StampPacket(PacketType::Message, p);
            std::string message;
            message = "Client left! " + l_ip.toString() +
                ":" + std::to_string(l_port);
            p << message;
            l_server->Broadcast(p, id);
        } else if ((PacketType)l_id == PacketType::Message){
            std::string receivedMessage;
            l_packet >> receivedMessage;
            std::string message = l_ip.toString() + ":" +
                std::to_string(l_port) + " : " + receivedMessage;
            sf::Packet p;
            StampPacket(PacketType::Message, p);
            p << message;
            l_server->Broadcast(p, id);
        }
    } else {
        if ((PacketType)l_id == PacketType::Connect){
            ClientID id = l_server->AddClient(l_ip, l_port);
            sf::Packet packet;
            StampPacket(PacketType::Connect, packet);
            l_server->Send(id, packet);
        }
    }
}
```

As we're going to be providing a pointer to this function to our `Server` class, the fingerprint has to match exactly. The function itself starts by first establishing whether the client ID for the provided IP address and port number exists. If it is indeed above or equal to zero, we're only interested in two types of packets: `Disconnect` and `Message`.

In a case of client disconnect, we create a message packet that is broadcasted to all clients except the one disconnecting. On the other hand, if there is an incoming message from one of the clients, it is first extracted and attached to a string that contains the client's IP address and port. We're not going to be using nicknames this time. The full message string is then attached to a message packet and broadcasted to every client, except the one that sent the message to begin with.

If a client wasn't found, however, all we care about is receiving connect packets. When one is received, the IP address and port are added and a connect packet is sent back to the client.

What kind of a server would it be if it didn't have the ability to process commands? Let's write a function that will be running in a separate thread and process user input:

```
void CommandProcess(Server* l_server){
    while (l_server->IsRunning()){
        std::string str;
        std::getline(std::cin, str);
        if (str == "!quit"){
            l_server->Stop();
            break;
        } else if (str == "dc"){
            l_server->DisconnectAll();
            std::cout << "DC..." << std::endl;
        } else if (str == "list"){
            std::cout << l_server->GetClientCount()
                << " clients online:" << std::endl;
            std::cout << l_server->GetClientList() << std::endl;
        }
    }
}
```

Note that `std::getline` is a blocking function. If the program stops running, the thread that this function is running on will still be blocking until some user input comes through. One way to get it to terminate is to implement a command that stops the server, which is what `!quit` does. Once the server `Stop` method is invoked, it also breaks the loop just to be safe.

The other two commands are pretty standard. One simply disconnects all users, while the other prints out a list of all connected clients. We haven't covered `GetClientCount` or `GetClientList`, since they're fairly basic and are not required for the server to run. You can find the implementation of these two methods in the source code of this book.

Now it's time to assemble and run our code:

```
int main() {
    Server server(handler);

    if (server.Start()) {
        sf::Thread c(&CommandProcess, &server);
        c.launch();

        sf::Clock clock;
        clock.restart();
        while (server.IsRunning()) {
            server.Update(clock.restart());
        }
        std::cout << "Stopping server..." << std::endl;
    }

    system("PAUSE");
    return 0;
}
```

This is quite a basic setup for the entry point of an application like this. First, we create an instance of the `Server` class and provide a function pointer that is going to be handling packets in its constructor. We then attempt to start the server and catch the return value of that in an `if` statement. If a successful start took place, a command thread is set up and launched. An instance of `sf::Clock` is created and restarted before entering our main loop, which simply executes as long as the server is running and updates it with the elapsed time value between iterations. That's all we need to have in a chat server!

The chat client

Our server is quite useless if we don't have the means of connecting to it and sending messages back and forth. In a separate project, let's create a file named `Client_Main.cpp` and begin writing the client portion of the code, starting with a packet handler:

```
void HandlePacket(const PacketID& l_id,
    sf::Packet& l_packet, Client* l_client)
{
    if ((PacketType)l_id == PacketType::Message) {
        std::string message;
        l_packet >> message;
        std::cout << message << std::endl;
    }
}
```

```

    } else if ((PacketType)l_id == PacketType::Disconnect){
        l_client->Disconnect();
    }
}

```

As you can see, it's really quite a simple design when we have a proper support class to fall back on. The client responds to two types of packets: messages and disconnects. In case a message pops in, it's extracted and simply printed in the console window. If a disconnect packet arrives from the server, the client's `Disconnect` method is invoked.

Next, the function that will be running in a command thread:

```

void CommandProcess(Client* l_client){
    while (l_client->IsConnected()){
        std::string str;
        std::getline(std::cin, str);
        if (str != ""){
            if (str == "!quit"){
                l_client->Disconnect();
                break;
            }
            sf::Packet p;
            StampPacket(PacketType::Message, p);
            p << str;
            l_client->Send(p);
        }
    }
}

```

We're using the same basic principle of capturing console input with the `std::getline` function, except we're only going to be processing the quit command in this case. Anything else that gets entered is treated as a message and sent out to the server. Keep in mind that because the `std::getline` function is a blocking one, a user will have to press enter once if the client gets disconnected by the server, just to provide some input and get things moving again in order for the thread to be closed.

Lastly, let's put all of this code to work and implement the main loop of our chat client:

```

void main(int argc, char** argv){
    sf::IpAddress ip;
    PortNumber port;
    if (argc == 1){
        std::cout << "Enter Server IP: ";
        std::cin >> ip;
    }
}

```

```
        std::cout << "Enter Server Port: ";
        std::cin >> port;
    } else if (argc == 3) {
        ip = argv[1];
        port = atoi(argv[2]);
    } else {
        return;
    }

    Client client;
    client.SetServerInformation(ip, port);
    client.Setup(&HandlePacket);
    sf::Thread c(&CommandProcess, &client);
    if (client.Connect()) {
        c.launch();
        sf::Clock clock;
        clock.restart();
        while (client.IsConnected()) {
            client.Update(clock.restart());
        }
    } else {
        std::cout << "Failed to connect." << std::endl;
    }
    std::cout << "Quitting..." << std::endl;
    sf::sleep(sf::seconds(1.f));
}
```

We begin by setting up a few variables to hold the IP address and port of the server. For extra credit, let's add support for command line arguments. If none are provided, the user is simply prompted to enter the server information inside the console window. Otherwise, the command-line arguments are read in and used for the same purpose.

Moving further, we see that an instance of `Client` is created and set up with the server information that was provided, in addition to the packet handler function getting registered and a command thread being prepared. The client then attempts to connect to the server, and if the connection is successful, the command thread is launched, an instance of `sf::Clock` is created, and the main loop of the program is entered, where the client gets updated.

With this, we have a fairly simple yet functional chat application:

```

C:\Repositories\SFML by example\Chapter 13\Debug\Server.exe
Incoming port: 5600. Outgoing port: 55282
Beginning to listen...
Re-try<3> heartbeat for client 0
Re-try<4> heartbeat for client 0
Re-try<5> heartbeat for client 0
Client 0 has timed out.
Re-try<3> heartbeat for client 1
Re-try<4> heartbeat for client 1
Re-try<5> heartbeat for client 1
Client 1 has timed out.

C:\Repositories\SFML by example\Chapter 1...
Enter Server IP: 127.0.0.1
Enter Server Port: 5600
Bound client to port: 49599
Attempting to connect to: 127.0.0.1:5600
Beginning to listen...
Hello! How are you?
127.0.0.1:58575 :I'm doing well, thank you!

C:\Repositories\SFML by example\Chapter ...
Enter Server IP: 127.0.0.1
Enter Server Port: 5600
Bound client to port: 58575
Attempting to connect to: 127.0.0.1:5600
Beginning to listen...
127.0.0.1:49599 :Hello! How are you?
I'm doing well, thank you!

```

Summary

Congratulations on making it this far! Having covered the most important fundamentals, which includes the basics of socket programming, utilization of threads, and building the underlying layer of client-server communications, we're finally ready to tackle actual game networking! In the final chapter of this book, we're going to integrate this network code into our existing code base, turning a lonely single player RPG into an exciting arena of player versus player combat! See you there!

